



Homework Assignment

Week 13: Functions Calling Functions

Total Points: 100 — Due: Next Class



Name: _____

Date: _____

Important Instructions

- Work on your own, but you may ask family members for hints only if stuck
- Write your code in Python (*using Thonny or any online IDE*)
- For setting up your code files, kindly see [instructions.pdf](#)
- For written answers, use the spaces provided on this sheet
- Show your work for full credit!

1 Part 1: Understanding Function Call Chains (15 points)

Time estimate: 20 minutes

1.1 Exercise 1.1: Code Tracing with Environment Diagrams (8 points)

Trace through this program and draw an environment diagram showing all frames:

```
1 def add_bonus(score):
2     return score + 10
3
4 def double(n):
5     return n * 2
6
7 def process_score(points):
8     step1 = double(points)
9     step2 = add_bonus(step1)
10    return step2
11
12 result = process_score(15)
13 print(result)
```

Your environment diagram:

Instructions:

- Draw frames for: Global, `process_score`, `double`, and `add_bonus`
- Show the order of function calls with numbered arrows
- Write the return value for each function
- Show which frame calls which other frame

Final output: _____

1.2 Exercise 1.2: Predict the Output (7 points)

What will each program print? Trace carefully through each function call:

Program A:

```

1 def helper(x):
2     return x + 5
3
4 def main_func(a, b):
5     result1 = helper(a)
6     result2 = helper(b)
7     return result1 + result2
8
9 print(main_func(10, 20))

```

Your prediction: _____

Show your work:

Program B:

```

1 def square(n):
2     return n * n
3
4 def sum_of_squares(a, b):
5     return square(a) + square(b)
6
7 def process(x, y, z):
8     part1 = sum_of_squares(x, y)
9     part2 = square(z)
10    return part1 + part2
11
12 print(process(2, 3, 4))

```

Your prediction: _____

Show your work:

2 Part 2: Scope and Variable Isolation (18 points)

Time estimate: 25 minutes

2.1 Exercise 2.1: Same Name, Different Variable (10 points)

Study this code carefully and answer the questions:

```

1 def outer():
2     x = 100
3     y = inner()
4     print(f"In outer: x = {x}")
5     return y
6
7 def inner():
8     x = 5
9     return x * 2
10
11 result = outer()
12 print(f"Result: {result}")

```

Questions:

1. How many different variables named `x` exist in this program? _____
2. What value does `x` have in `outer()` after `inner()` returns? _____
3. What does `inner()` return? _____
4. What will the program print? Write both lines:
Line 1: _____
Line 2: _____
5. Explain why the `x` in `inner()` doesn't change the `x` in `outer()`:

2.2 Exercise 2.2: Multiple Choice - Scope Rules (8 points)

Circle the correct answer:

1. What happens when a function finishes executing?
 - A. Its local variables are saved for later use
 - B. Its local variables are destroyed
 - C. Its local variables become global
 - D. Its local variables are copied to the calling function
2. How can a function share data with another function?
 - A. By using the same variable names
 - B. Through parameters and return values
 - C. Local variables are automatically shared
 - D. By declaring variables as global
3. What will this code print?

```

1 def func_a():
2     num = 10
3     return func_b()
4
5 def func_b():
6     num = 20
7     return num
8
9 print(func_a())

```

- A. 10
- B. 20
- C. 30
- D. Error - num is not defined

4. In a chain of function calls $A \rightarrow B \rightarrow C$, which function's frame exists first?

- A. Function A's frame
- B. Function B's frame
- C. Function C's frame
- D. All frames are created at the same time

3 Part 3: Code Completion - Building Function Chains (20 points)

Time estimate: 25 minutes

3.1 Exercise 3.1: Complete the Helper Functions (12 points)

Open [week13_ex3.1.py](#) and complete this grade processing program:

```

1 # Grade Processing System with Function Chains
2
3 def calculate_average(grades):
4     """Calculate and return the average of a list of grades."""
5     total = -----
6     count = -----
7     return -----
8
9 def get_letter_grade(score):
10    """Convert numeric score to letter grade."""
11    if score >= 90:
12        return -----
13    elif score >= 80:
14        return -----
15    elif score >= 70:
16        return 'C'
17    elif score >= 60:
18        return 'D'
19    else:
20        return 'F'

```

```

21
22 def get_status(letter):
23     """Determine pass/fail status from letter grade."""
24     if letter in ['A', 'B', 'C', 'D']:
25         return -----
26     else:
27         return -----
28
29 def process_student(name, grades):
30     """Process a student's grades - calls all helper functions."""
31     avg = ----- # Call calculate_average
32     letter = ----- # Call get_letter_grade
33     status = ----- # Call get_status
34     return f"{name}: {avg:.1f} ({letter}) - {status}"
35
36 # Test the function chain
37 student_grades = [85, 92, 78, 88, 95]
38 print(process_student("Ahmed", student_grades))
39
40 # More tests
41 print(process_student("Sara", [70, 65, 72, 68]))
42 print(process_student("Ali", [45, 55, 50, 48]))

```

3.2 Exercise 3.2: Build a Calculation Chain (8 points)

Open [week13_ex3.2.py](#) and complete this program:

```

1 # Price Calculator with Multiple Functions
2
3 def apply_discount(price, percent):
4     """Apply a percentage discount to a price."""
5     discount_amount = price * (percent / 100)
6     return -----
7
8 def add_tax(price, tax_rate):
9     """Add tax to a price."""
10    tax_amount = -----
11    return price + tax_amount
12
13 def calculate_final_price(original, discount_percent, tax_percent):
14     """Calculate final price after discount and tax.
15     First apply discount, then add tax to discounted price."""
16     after_discount = -----
17     final = -----
18     return final
19
20 # Test the calculation chain
21 original_price = 1000
22 discount = 20 # 20% off
23 tax = 17 # 17% sales tax
24
25 final = calculate_final_price(original_price, discount, tax)
26 print(f"Original: Rs. {original_price}")
27 print(f"After {discount}% discount and {tax}% tax: Rs. {final:.2f}")
28
29 # Expected: 1000 - 200 = 800, then 800 + 136 = 936

```

4 Part 4: Code Ordering - Function Structure (12 points)

Time estimate: 15 minutes

4.1 Exercise 4.1: Put the Functions in Order (12 points)

These code blocks create a complete password validator program. Number them in the correct order (1-8):

Order	Code Block
--	<pre>def main(): password = input("Enter password: ") if validate_password(password): print("Password accepted!") else: print("Password rejected.")</pre>
--	<pre>def has_digit(text): for char in text: if char.isdigit(): return True return False</pre>
--	<pre>def is_long_enough(text): return len(text) >= 8</pre>
--	<pre>main()</pre>
--	<pre>def validate_password(password): if not is_long_enough(password): return False if not has_digit(password): return False return True</pre>

Which function calls which? Draw arrows or list the relationships:

5 Part 5: Debugging Multi-Function Programs (15 points)

Time estimate: 20 minutes

Each snippet has bugs. Find and fix ALL errors:

5.1 Snippet 1: Missing Return Value (8 points)

```

1 def get_max(numbers):
2     max_val = numbers[0]
3     for num in numbers:
4         if num > max_val:
5             max_val = num
6         # Bug: forgot to return!
7
8 def get_min(numbers):
9     min_val = numbers[0]
```

```

10     for num in numbers:
11         if num < min_val:
12             min_val = num
13     return min_val
14
15 def get_range(numbers):
16     maximum = get_max(numbers)
17     minimum = get_min(numbers)
18     return maximum - minimum # Bug: maximum is None!
19
20 data = [5, 2, 8, 1, 9, 3]
21 print(f"Range: {get_range(data)}")

```

What's wrong?

Fixed version:

```

1 # Write the corrected get_max function:

```

Save your corrected code as `week13_ex5_1_fixed.py`

5.2 Snippet 2: Wrong Function Call Order (7 points)

```

1 def format_name(first, last):
2     full = combine_names(first, last)
3     return full.upper()
4
5 def combine_names(a, b):
6     return a + " " + b
7
8 def create_greeting(first_name, last_name):
9     # Bug: arguments in wrong order!
10    formatted = format_name(last_name, first_name)
11    return f"Hello, {formatted}!"
12
13 print(create_greeting("Muhammad", "Ali"))
14 # Expected: "Hello, MUHAMMAD ALI!"
15 # Actual: "Hello, ALI MUHAMMAD!"

```

What's wrong?

Fixed line:

formatted = _____

Save your corrected code as `week13_ex5_2_fixed.py`

6 Part 6: Full Program - Quiz Game (25 points)

Time estimate: 40 minutes

6.1 Exercise 6.1: Build a Complete Quiz Game (25 points)

Create a program called `quiz_game.py` using the main function pattern:

Program Requirements

Your program should have these functions:

- `create_questions()` - Returns a list of question dictionaries
- `ask_question(question_dict)` - Asks one question, returns True/False
- `calculate_percentage(correct, total)` - Returns percentage score
- `get_feedback(percentage)` - Returns feedback message based on score
- `run_quiz(questions)` - Runs the quiz, returns number correct
- `main()` - Coordinates the entire program

Each function should do ONE thing and call other functions when needed.

Example Run:

```
=== Welcome to the Python Quiz! ===
```

Question 1: What keyword starts a function definition?

Your answer: def

Correct!

Question 2: What data structure uses key-value pairs?

Your answer: dictionary

Correct!

Question 3: What loop runs while a condition is True?

Your answer: for

Incorrect! The answer was: while

```
=== Quiz Complete! ===
```

Score: 2/3 (66.7%)

Feedback: Good job! Keep practicing!

Starter Structure:

```
1 # quiz_game.py
2 # Name: -----
3 # Date: -----
4
5 def create_questions():
6     """Create and return a list of quiz questions."""
7     questions = [
8         {"question": "What keyword starts a function?",
9          "answer": "def"},
10        {"question": "What data structure uses key-value pairs?",
11         "answer": "dictionary"},
12        {"question": "What loop runs while a condition is True?",
13         "answer": "while"},
14        # Add 2 more questions of your own!
15        -----
16        -----
17    ]
18    return questions
19
```

```

20 def ask_question(q_dict):
21     """Ask a single question and return True if correct."""
22     print(q_dict["question"])
23     user_answer = input("Your answer: ").lower().strip()
24     correct_answer = q_dict["answer"].lower()
25
26     if user_answer == correct_answer:
27         print("Correct!\n")
28         return -----
29     else:
30         print(f"Incorrect! The answer was: {correct_answer}\n")
31         return -----
32
33 def calculate_percentage(correct, total):
34     """Calculate and return the percentage score."""
35     return -----
36
37 def get_feedback(percentage):
38     """Return appropriate feedback based on percentage."""
39     if percentage >= 80:
40         return "Excellent work! You're a Python pro!"
41     elif percentage >= 60:
42         return -----
43     else:
44         return -----
45
46 def run_quiz(questions):
47     """Run through all questions and return score."""
48     correct_count = 0
49     question_num = 1
50
51     for q in questions:
52         print(f"Question {question_num}: ", end="")
53         if -----: # Call ask_question
54             correct_count += 1
55             question_num += 1
56
57     return correct_count
58
59 def main():
60     """Main function - coordinates the quiz program."""
61     print("=== Welcome to the Python Quiz! ===\n")
62
63     # Get the questions
64     questions = -----
65
66     # Run the quiz
67     score = -----
68     total = len(questions)
69
70     # Calculate and display results
71     percentage = -----
72     feedback = -----
73
74     print("=== Quiz Complete! ===")
75     print(f"Score: {score}/{total} ({percentage:.1f}%)")
76     print(f"Feedback: {feedback}")
77
78     # Run the program
79     main()

```

Testing Checklist:

- ☐ Program has at least 6 functions
- ☐ Each function does ONE specific task
- ☐ Functions call other functions appropriately
- ☐ `main()` coordinates everything
- ☐ Quiz has at least 5 questions
- ☐ Score and percentage display correctly
- ☐ Feedback changes based on score
- ☐ Program handles lowercase/uppercase answers

7 Part 7: Reflection Questions (5 points)

Time estimate: 10 minutes

Answer these questions in complete sentences:

1. **Why is it better to have many small functions instead of one big function? Give a specific example from this homework.**

2. **Explain what happens to local variables when a function finishes executing. Why is this useful?**

3. **Looking back at Module 2, what was the most challenging concept you learned? What helped you understand it?**

8 Bonus Section: Extra Challenges (Optional - 10 extra points)

Only attempt if you've finished everything else!

8.1 Challenge 1: Function Composition (5 points)

Open [week13_bonus1.py](#) and create a text processing pipeline:

```

1 # Text Processing Pipeline
2
3 def remove_spaces(text):
4     """Remove all spaces from text."""

```

```

5     # Your code here
6     pass
7
8 def make_uppercase(text):
9     """Convert text to uppercase."""
10    # Your code here
11    pass
12
13 def add_borders(text):
14     """Add stars around text: ***text***"""
15     # Your code here
16     pass
17
18 def process_text(text):
19     """Apply all transformations in order."""
20     step1 = remove_spaces(text)
21     step2 = make_uppercase(step1)
22     step3 = add_borders(step2)
23     return step3
24
25 # Test
26 original = "hello world"
27 result = process_text(original)
28 print(result) # Should print: ***HELLOWORLD***
29
30 # Add your own transformation function and include it!

```

8.2 Challenge 2: Calculator with Functions (5 points)

Create a program called `calculator.py` that:

- Has separate functions for add, subtract, multiply, divide
- Has a `get_operation()` function that returns the correct function
- Has a `calculate(a, op, b)` function that calls the right operation
- Uses `main()` to run a simple calculator interface

Example:

```

Enter first number: 10
Enter operation (+, -, *, /): *
Enter second number: 5
Result: 50

```

9 Debugging Discoveries (5 bonus points)

Growth Mindset: Errors are learning opportunities!

Did you encounter any interesting errors while completing this homework? Share one error and how you fixed it:

🔧 My Debugging Story

The Error I Got: _____

What I Tried First: _____

How I Fixed It: _____

What I Learned: _____

No errors? Share a tricky part you figured out instead!

Submission Checklist

Before submitting, make sure you have:

- ☐ Completed all required sections (Parts 1-7)
- ☐ Drawn environment diagrams for Part 1
- ☐ Saved each code exercise in its own file:
 - `week13_ex3_1.py`
 - `week13_ex3_2.py`
 - `week13_ex5_1_fixed.py`
 - `week13_ex5_2_fixed.py`
 - `quiz_game.py`
- ☐ Created a folder named `week13_hw_yourname` with all files
- ☐ Tested each code file individually to ensure it runs without errors
- ☐ Verified that functions call each other correctly
- ☐ Checked that all functions have proper return statements
- ☐ Confirmed the main function pattern is used in full programs
- ☐ Written your name at the top of this paper
- ☐ Answered all reflection questions
- ☐ Attempted bonus section (optional)

Module 2 Final Check:

Rate your confidence with functions (1-5): _____

Which Module 2 topic do you feel strongest in? _____

Time Tracking:

How long did this homework take you? _____ hours

Parent/Guardian Signature: _____

(Confirming student completed work independently)

Congratulations on completing Module 2!

You've mastered data structures and functions - you're thinking like a real programmer!

Next module: Object-Oriented Programming awaits!